Final Project Topic Catalog

Advanced Database Systems (Polyglot Persistence)

Faculty of Information Technology

Semester II, 2025–2026

Software Support: Oracle Database, MS SQL Server, MongoDB Atlas

Table of Contents & Topic Selection Guide

#	Topic Name	SQL Tech	NoSQL Tech	Diff.
1	Global FinTech Ledger	Sagas/Locks	Data Locality	High
2	Flash Sale Marketplace	Serializability	AP Strategy	High
3	P2P Micro-Lending	Recursive CTE	Risk Signals	Med
4	Crypto Wallet	Quorum Writes	LSM-Tree Ingest	High
5	Smart City Dashboard	Hierarchy CTE	TTL/Pruning	Med
6	Supply Chain Telemetry	Custody Tx	Path Traversal	High
7	Ride-Sharing Fleet	Pessimistic Lock	Geo-Spatial	Med
8	Smart Grid Controller	WAL/Durability	Aggregation	Med
9	Unified E-Health	Staff Schedules	Schema Evolution	High
10	DevConnect Social	Social Graph	Full-Text Search	Med
11	HR Cloud Platform	Org Charts	Nested Resumes	Med
12	Esports Analytics	Tie-Handling	Match Timeline	Med
13	Streaming Progress	Left Join/Churn	Eventual Cons.	Med
14	Hotel Reservation	Price Triggers	Search-on-Read	High
15	Research Library	Citation Chain	Author Arrays	Med
16	Airline Loyalty	Compensating Tx	Delay Logs	High
17	Real Estate Escrow	Status Triggers	Multimedia Specs	Med
18	E-Learning Analytics	Percentile Rank	Fail Rate Agg.	Med
19	Charity Crowdfunding	Tax Receipt Trg	Threaded Comm.	Med
20	MMO Game Economy	2PC/Duplication	Variety Attributes	High

Topic 1: Global FinTech Ledger & Fraud Analytics

Advanced Database Systems Final Project Brief

Business Context: The Source of Truth

In international banking, a "Ledger" is the digital source of truth. When a customer sends money across borders, the system must guarantee funds are never lost and double-spending is impossible. Banks also analyze IP addresses and device fingerprints to detect hackers.

User & Customer Requirements

Customers expect fund transfers to be immediate or fail with a full refund. They expect to be notified if their account is accessed from an unusual location. Business managers need a report showing the total financial exposure of "Family Groups" where one parent manages multiple sub-accounts.

Project Requirements & Instructional Guidance

- The technical team must implement a multi-table SQL transaction in **Oracle or MS SQL Server** using **Pessimistic Locking** to ensure balance integrity.
- A **Database Trigger** must automatically record every balance change into an immutable audit table.
- Use a **Recursive CTE** to traverse the family account hierarchy and calculate aggregate balances.
- High-velocity login metadata must be stored in **MongoDB** to allow for flexible device data.

Database Schema & Architectural Split

SQL: `Accounts(ID, Balance, Status)`, `AuditLog(ID, AccID, OldVal, NewVal, Time)`.
NoSQL: `AccessEvents { user_id, timestamp, metadata: { ip, device, location } }`.

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 2: High-Concurrency Flash Sale Marketplace

Advanced Database Systems Final Project Brief

Business Context: The Thunderous Herd

Flash sales create massive concurrent traffic. If stock levels are not managed with strict locking, the system might over-sell items. Shopping carts must be Highly Available so users don't face errors while browsing during peak load.

User & Customer Requirements

Shoppers want to add items to their cart instantly without the website "freezing." During checkout, they expect a 100% guarantee that their order is confirmed only if stock is available. Logistics managers need a real-time leaderboard ranking the districts with the highest demand.

Project Requirements & Instructional Guidance

- Technical implementation must include a **Trigger** in SQL that validates stock levels *before* an update, raising an error to prevent negative stock.
- Use **Serializable Isolation** during final payment to prevent the "Lost Update" anomaly.
- Temporary shopping carts should be stored in **MongoDB** using an **AP strategy**.
- Use SQL **Window Functions** to generate a leaderboard of highest-selling products per region.

Database Schema & Architectural Split

SQL: `Inventory(ProdID, StockCount), Orders(OrderID, UserID, Total, Status).`
NoSQL: `ShoppingCarts { session_id, items: [ {prod_id, qty} ], last_active }.`

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 3: P2P Micro-Lending & Credit Scoring

Advanced Database Systems Final Project Brief

Business Context: Alternative Trust

Micro-lending uses "Alternative Data" (app usage, social signals) to assess risk for people without credit scores. The business relies on the precise calculation of interest and strict enforcement of repayment schedules.

User & Customer Requirements

Borrowers require a transparent 12-month schedule showing principal and interest. Investors demand a "Risk Score" updated dynamically based on behavior. The business must prevent "Double Borrowing" where users take new loans while current ones are unpaid.

Project Requirements & Instructional Guidance

- The team must use a **Recursive CTE** in SQL to generate an automated amortization schedule for each loan.
- A SQL **Stored Procedure** must handle loan disbursement, checking eligibility atomically.
- Ingest unstructured behavioral signals into **MongoDB** for flexible risk modeling.
- Use a MongoDB **Aggregation Pipeline** to calculate borrower repayment speed percentiles.

Database Schema & Architectural Split

SQL: `Loans(ID, BorrowerID, Principal, Rate), Payments(ID, LoanID, Amount, Date).`
NoSQL: `UserSignals { user_id, signals: [ {type: "app_usage", score: 0.9} ] }.`

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 4: Multi-Asset Crypto Exchange Wallet

Advanced Database Systems Final Project Brief

Business Context: 24/7 Digital Markets

A crypto exchange matches buyers and sellers globally. The "Hot Wallet" stores active balances and must be protected from double-spending. The "Order Book" fluctuations happen every millisecond and require extreme ingestion speed.

User & Customer Requirements

Traders expect orders paired at sub-second speeds. Users require withdrawals protected by strict transactional boundaries and daily limits. The business needs a "Market Depth" report showing order volumes at every price point.

Project Requirements & Instructional Guidance

- The technical team must prioritize **Consistency (CP)** in SQL for fund movements using row-level locking.
- Implement a SQL **Trigger** that enforces a rolling 24-hour withdrawal limit for every user account.
- Store high-frequency trade history in **MongoDB**, explaining the use of an **LSM-Tree** approach.
- Use MongoDB **Aggregation** to calculate real-time "Price Spread" across currency pairs.

Database Schema & Architectural Split

SQL: `Wallets(UserID, Asset, Bal), Withdrawals(ID, UserID, Amt, Timestamp).`
NoSQL: `OrderBook { pair: "BTC/USDT", orders: [ {price, qty, side, user} ] }.`

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 5: Smart City IoT Sensor Dashboard

Advanced Database Systems Final Project Brief

Business Context: The Urban Nervous System

Smart cities use sensors to monitor noise and air quality. The system must manage a physical registry (hardware locations) and a telemetry stream (readings every 5 seconds). Maintaining these assets is as critical as the data itself.

User & Customer Requirements

City officials want an organizational tree of sensors sorted by District to manage maintenance crews. Citizens want a leaderboard of "Cleanest Neighborhoods." The team requires the system to automatically purge data older than 30 days.

Project Requirements & Instructional Guidance

- Use a **Recursive CTE** in SQL to model the city's geographical hierarchy from City to Sensor.
- Ingest telemetry into **MongoDB** using **TTL Indexes** to automate data pruning.
- Use SQL **Window Functions** to calculate moving CO2 averages for every ward.
- Explain the strategy used to manage **Write Amplification** on the NoSQL layer.

Database Schema & Architectural Split

SQL: `SensorRegistry(ID, LocationID, Type), Locations(ID, Name, ParentID).`
NoSQL: `Telemetry { sensor_id, timestamp, data: { co2, noise }, _expiry }.`

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 6: Global Supply Chain & Asset Telemetry

Advanced Database Systems Final Project Brief

Business Context: Chain of Custody

Tracking containers involves complex legal hand-offs. Each hand-off is a "Change of Custody." If a sensor shows vaccines became too warm, the system must alert the owner and flag the shipment for inspection immediately.

User & Customer Requirements

Owners want to see the exact path their container took, including temperature logs. Logistics providers need alerts if a container has not checked into a port within 12 hours. The system must support various sensor types without schema changes.

Project Requirements & Instructional Guidance

- Implement the "Change of Custody" as an atomic **SQL Transaction** to ensure ownership is never ambiguous.
- Use a **Graph-style lookup** in MongoDB to find the quickest historical route between ports.
- Create a SQL **Trigger** that updates shipment status to "ALARM" if NoSQL telemetry indicates violation.
- Defend the NoSQL Document model choice using the concept of **Schema-on-Read**.

Database Schema & Architectural Split

SQL: `Shipments(ID, Status), Ownership(ID, ShipmentID, PartyID, Time).`
NoSQL: `TelemetryLogs { shipment_id, route, logs: [ {lat, lng, temp, t} ] }.`

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 7: Ride-Sharing Fleet Management

Advanced Database Systems Final Project Brief

Business Context: Real-time Dispatching

Ride-sharing matching is a high-concurrency event. When a rider requests a car, the system must find the nearest driver, lock that driver atomically, and calculate a surge price based on current local demand.

User & Customer Requirements

Riders want a driver in under 5 seconds. Drivers expect accurate earnings and want to see performance rankings. The business must guarantee no driver is assigned to two riders simultaneously, maintaining matching consistency.

Project Requirements & Instructional Guidance

- Use **Pessimistic Locking** (`FOR UPDATE`) in SQL on the `Drivers` table during matching.
- Store GPS coordinates in **MongoDB** using **Geo-spatial Indexes** for efficient radius queries.
- Use the **Saga Pattern** to manage the trip life-cycle and implement compensating transactions for payment failure.
- Use SQL **DENSE_RANK** to create a performance leaderboard of drivers within city zones.

Database Schema & Architectural Split

SQL: `Drivers(ID, Status, Rating)`, `Trips(ID, DriverID, Price, Time)`.
NoSQL: `LiveMap { driver_id, loc: { type: "Point", coordinates: [lng, lat] } }`.

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 8: National Smart Energy Grid Controller

Advanced Database Systems Final Project Brief

Business Context: Smart Metering

National power grids use Smart Meters to report consumption every 15 minutes. This allows for detection of blackouts or theft. The system requires a hybrid architecture: a strict SQL ledger and a high-volume NoSQL layer for readings.

User & Customer Requirements

Consumers want to see hourly usage history. Grid operators need to identify stress areas in real-time. Financial auditors require that every monthly bill is generated with 100% precision recorded against the legal contract.

Project Requirements & Instructional Guidance

- Implement a SQL **Trigger** that flags "Critical Priority" customers if their power usage drops to zero.
- Students must justify why an **LSM-Tree** based engine is the best choice for storing billions of readings.
- Use a MongoDB **Aggregation Pipeline** to detect consumption anomalies vs neighborhood averages.
- Explain how **Write-Ahead Logging (WAL)** ensures billing transactions survive power failure.

Database Schema & Architectural Split

SQL: `Customers(ID, Priority, Contract)`, `Invoices(ID, CustID, Month, Amt)`.
NoSQL: `UsageReadings { meter_id, readings: [ {usage, timestamp} ] }`.

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 9: Unified E-Health Patient Records

Advanced Database Systems Final Project Brief

Business Context: Diversity in Medicine

Patient data ranges from numeric blood tests to images and text notes. While history must be flexible, scheduling must be perfectly consistent. Double-booking a surgeon is a failure that endangers lives and reputations.

User & Customer Requirements

Patients want their entire history across clinic in one view. Administrators need to prevent overbooking and ensure procedures are logged. Researchers want to identify "Genetic Clusters" to study hereditary health risks.

Project Requirements & Instructional Guidance

- Technical implementation must include a SQL **Stored Procedure** with strict isolation to manage schedules.
- Implement **Schema Evolution** in NoSQL to handle adding vaccination fields to old records.
- Use a **Recursive CTE** in SQL to traverse family history and identify relatives of a specific patient.
- Create a MongoDB **Aggregation Pipeline** to filter through patient records by diagnosis.

Database Schema & Architectural Split

SQL: `Staff(ID, Role), Appts(ID, StaffID, PatientID, Start, End).`
NoSQL: `Histories { patient_id, events: [ {type: "lab", result: 95}, {type: "note"} ] }.`

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 10: DevConnect: Social Platform for Developers

Advanced Database Systems Final Project Brief

Business Context: The Social Graph

DevConnect manages a complex "Social Graph" of developers. It hosts an activity feed where users post code and get real-time feedback. Security and permission management are vital to ensuring code ownership.

User & Customer Requirements

Users want suggestions for "Developers you may know" based on mutual connections. Developers want to search for code snippets using tags. Platform owners need to rank the "Most Influential Snippets" over the last 7 days.

Project Requirements & Instructional Guidance

- Implement a **Recursive CTE** in SQL to find mutual connections up to the 3rd degree.
- Use a NoSQL Document Store for code snippets, utilizing a **$text index** for bios and tags.
- Explain **Impedance Mismatch** by comparing SQL JOINs vs. a single NoSQL document read for retrieval.
- Use SQL **Window Functions** to rank the most popular posts over a rolling window.

Database Schema & Architectural Split

SQL: `Users(ID, Username)`, `Following(FollowerID, FolloweeID, Since)`.
NoSQL: `Posts { user_id, snippet: "...", tags: [ "sql", "react" ], likes: 120 }`.

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 11: Global Recruitment & HR Cloud Platform

Advanced Database Systems Final Project Brief

Business Context: Human Capital

Managing employees requires precision in payroll. Resumes come in diverse formats and must be ingested into a flexible search layer. Multi-step approval chains depend on a deep organizational hierarchy.

User & Customer Requirements

Employees want to see their full chain of command. Managers want to approve leave, expecting the system to automatically deduct days from SQL balances. Recruiters want to find candidates with specific nested skills.

Project Requirements & Instructional Guidance

- Use a **Recursive CTE** to generate the complete organizational chart from the CEO down.
- Implement a SQL **Stored Procedure** for "Leave Approval" that atomically checks balances.
- Store resumes in MongoDB to handle the **Variety** of candidate data using nested objects.
- Create a MongoDB **Aggregation Pipeline** to filter candidates based on complex nested criteria.

Database Schema & Architectural Split

SQL: `Staff(ID, ManagerID, Salary), LeaveRecords(ID, StaffID, Days).`
NoSQL: `Resumes { candidate_id, experience: [ {company, role, years} ], skills: {...} }.`

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 12: Esports League Analytics & Leaderboards

Advanced Database Systems Final Project Brief

Business Context: Competitive Data

Professional Esports generates billions of data points. League managers need to track team rosters and prize money. Fans demand real-time rankings based on complex metrics like KDA and Gold-per-Minute.

User & Customer Requirements

Fans expect a global leaderboard that handles ties correctly. Managers must ensure a player is not registered on two team rosters in the same season. Analysts want to see average metrics over the last 50 games.

Project Requirements & Instructional Guidance

- Use SQL **DENSE_RANK** to create a global player leaderboard that manages ties.
- Implement a SQL **Trigger** that validates roster constraints *before* insertion into the `Rosters` table.
- Ingest nested match logs into MongoDB for player-specific timelines compared to flat tables.
- Use a MongoDB **Aggregation Pipeline** to calculate player averages by unwinding logs.

Database Schema & Architectural Split

SQL: `Teams(ID, Name), Players(ID, Nickname), Rosters(PlayerID, TeamID).`
NoSQL: `MatchStats { match_id, teams: [ {team_id, players: [ {id, kills} ] } ] }.`

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 13: High-Scale Content Streaming

Advanced Database Systems Final Project Brief

Business Context: The Streaming Engine

Streaming services must manage billing (Strict Consistency) and watch progress (High Availability). Expiration must be handled precisely. Content uses thousands of flexible discovery tags.

User & Customer Requirements

Users expect "Continue Watching" to update instantly across devices. Business owners want to identify "Churning Users" who haven't logged in for 30 days to target them with marketing.

Project Requirements & Instructional Guidance

- Technical team must use SQL to manage subscriptions, defending the use of ACID properties.
- Implement "Watch Progress" sync in NoSQL using an **AP strategy** and handle "Read-Your-Writes."
- Write an advanced SQL query using a **LEFT JOIN** to identify "Churning Users."
- Store movie metadata in NoSQL and handle varied content tags using Document models.

Database Schema & Architectural Split

SQL: `Users(ID, Email), Subscriptions(UserID, Plan, ExpiryDate).`
NoSQL: `UserProgress { user_id, movie_id, timestamp_sec, device_id }.`

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 14: Global Hotel Reservation Engine

Advanced Database Systems Final Project Brief

Business Context: Inventory & Amenities

Hotel booking involves search (Read-heavy) and booking (Write-heavy). If two users book the same last room, the system must resolve it atomically. Descriptions of amenities change as resorts upgrade.

User & Customer Requirements

Travelers want extremely fast searches. However, during payment, they demand a guaranteed room. Hotel managers want a report of the "Top 3 Revenue Generating Rooms" in their property per quarter.

Project Requirements & Instructional Guidance

- Implement a **Pessimistic Locking** strategy in SQL during booking to prevent double-booking.
- Use NoSQL for the "Hotel Catalog," prioritizing **Availability (AP)** for amenity searches.
- Create a SQL **Trigger** that notifies a log table whenever a room rate is changed by $>$50%.
- Use SQL **Window Functions** to rank room performance within each hotel based on revenue.

Database Schema & Architectural Split

SQL: `Hotels(ID, City), Rooms(ID, HotelID, Rate, Status), Bookings(ID, RoomID).`
NoSQL: `HotelCatalog { hotel_id, description, amenities: [ "pool" ], images: [ ... ] }.`

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 15: Digital Research Library & Citations

Advanced Database Systems Final Project Brief

Business Context: Academic Networks

Scientific knowledge is built on a citation network. Researchers use this to find influential work and track impact. The system must manage a hierarchical taxonomy and store metadata for millions of papers.

User & Customer Requirements

Researchers want to find the complete "Citation Chain" for fundamental papers. Library managers need to know which authors are most cited. System must ensure strict ACID for institutional access rights.

Project Requirements & Instructional Guidance

- Implement a **Recursive CTE** in SQL to traverse citation graph descendants of a specific paper.
- Use NoSQL Document Store for metadata and an **Aggregation Pipeline** to calculate citation counts.
- Explain the team handles **Impedance Mismatch** when storing author arrays in NoSQL.
- Use SQL for membership management to ensure integrity for institutional access rights.

Database Schema & Architectural Split

SQL: `Papers(ID, Title, Year), Citations(SourceID, TargetID).`
NoSQL: `PaperDetails { paper_id, authors: [ {id, name} ], abstract, tags }.`

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 16: Airline Reservation & Loyalty System

Advanced Database Systems Final Project Brief

Business Context: High-Stakes Seating

Airline seats are high-stakes. Loyalty points act like a second currency and must be deducted atomically with ticket issuance. Airline tracks customer preferences and flight delay logs using flexible models.

User & Customer Requirements

Passengers expect loyalty point refunds if flights are canceled. Gate agents need priority lists for upgrades based on tiers. Businesses need reports on "Most Delayed Routes" to negotiate with airports.

Project Requirements & Instructional Guidance

- Implement a **Saga Pattern** for the booking process with compensating transactions to refund points.
- Use a SQL **Trigger** to ensure `MilesBalance` never drops below zero during redemption.
- Store flight delay logs in **MongoDB**, justifying why Document models are better for varied logs.
- Use SQL **Window Functions** to rank airline routes by average delay time.

Database Schema & Architectural Split

SQL: `Users(ID, Tier, Miles)`, `Seats(FlightID, SeatNum, Status)`, `Tickets(ID, UserID)`.
NoSQL: `FlightLogs { flight_id, delays: [ {reason, mins, report} ] }`.

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 17: Real Estate Escrow & Listing Platform

Advanced Database Systems Final Project Brief

Business Context: Trust & Property

Escrow holds funds until ownership is transferred. Sale status must be 100% consistent. Listings are varied, containing rich media and flexible attributes like solar specifications or acreage.

User & Business Requirements

Buyers expect "Sold" status updates instantly upon closing. Agents need reports on their "Sales Percentile" in the city. Legal auditors need an immutable trail of ownership changes for compliance verification.

Project Requirements & Instructional Guidance

- Implement a SQL **Trigger** that moves property status to "PENDING" automatically when deposit is recorded.
- Use NoSQL for listings and demonstrate **Schema Evolution** by adding new media fields to records.
- Justify why "Price" can be eventually consistent while "Ownership" must be strongly consistent.
- Use SQL **Window Functions** to calculate percentile ranks of agents based on sales value.

Database Schema & Architectural Split

SQL: `Properties(ID, OwnerID, Status), Escrow(ID, PropID, Bal), Agents(ID, Name).`
NoSQL: `Listings { property_id, price, features: { solar: true }, photos: [ ... ] }.`

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 18: E-Learning Analytics (LMS Pro)

Advanced Database Systems Final Project Brief

Business Context: Adaptive Learning

Learning platforms track student interactions. Quizzes range from multiple choice to complex coding. System must identify "At-Risk" students in the bottom 10% so instructors can provide extra help.

User & Business Requirements

Students want to see their rank in the class. Teachers need to know quiz questions failure rates. University administrators need hierarchical reports showing performance from Faculty down to Sections.

Project Requirements & Instructional Guidance

- Use a **Recursive CTE** to model the academic hierarchy and aggregate metrics at every level.
- Utilize SQL **Window Functions** to identify students in the bottom 10th percentile of grades per section.
- Store quiz attempts in MongoDB and use an **Aggregation Pipeline** to calculate failure rates per question.
- Explain the **N+1 Problem** that occurs if application loops to fetch individual student grades.

Database Schema & Architectural Split

SQL: `Students(ID, Name), Grades(ID, StudentID, CourseID, Score).`
NoSQL: `QuizAttempts { user_id, quiz_id, answers: [ {q: 1, val: "A"} ] }.`

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 19: Charity Crowdfunding & Donor Ledger

Advanced Database Systems Final Project Brief

Business Context: Financial Transparency

Every dollar must be tracked to a specific campaign. Donors want to see real-time progress toward goals. Comments and stories are unstructured data requiring high locality for fast reading.

User & Business Requirements

Donors want to see their name on "Top Supporters" lists. Managers need automatic tax receipt generation for large donations. Site visitors expect to read threaded comments without page lag.

Project Requirements & Instructional Guidance

- Implement a SQL **Trigger** that generates a tax receipt record whenever a donation exceeds $50.
- Use SQL **Window Functions** to calculate the "Running Total" of donations for each campaign over time.
- Store threaded comments in MongoDB to exploit **Data Locality** for fast retrieval.
- Use MongoDB **Aggregation** to identify the most active commenters across all campaigns.

Database Schema & Architectural Split

SQL: `Donations(ID, CampID, DonorID, Amt, Time), Receipts(ID, DonID).`
NoSQL: `Campaigns { id, title, comments: [ {user, text, replies: [...]} ] }.`

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Topic 20: MMO Game Economy Marketplace

Advanced Database Systems Final Project Brief

Business Context: Digital Scarcity

In MMO games, player trades are critical. Trading requires two player inventories to be updated simultaneously. Item metadata is complex, with magic variations and durability stats changing with use.

User & Business Requirements

Players want to trade items safely and check their total "Net Worth." Developers need to prevent duplication bugs if network fails midway. Analysts want to rank wealthiest players in each guild.

Project Requirements & Instructional Guidance

- The backend must use **Serializable Isolation** for player trades in SQL to ensure ownership changes are atomic.
- Store item metadata in **MongoDB** to handle the Variety of item types without costly migrations.
- Use SQL **Window Functions** to calculate a player's "Net Worth" percentile within their specific guild.
- Implement a **2-Phase Commit (2PC)** strategy to ensure trade either completes fully or fails entirely.

Database Schema & Architectural Split

SQL: `Inventories(ID, PlayerID, ItemID, Status), Gold(PlayerID, Balance).`
NoSQL: `ItemDefinitions { id, name, stats: { dmg: 50 }, sockets: [ ... ] }.`

Evaluation Rubric (Individual Defense)

Criteria	Weight	Core Expectation
Arch. Defense	35%	Individual justification of SQL/NoSQL split and CAP choice.
Technical Logic	35%	Correctness of Triggers, Recursive CTEs, and Aggregations.
AI Audit & Ethics	15%	Quality of AI prompt documentation and manual code optimization.
Oral Defense	15%	Ability to modify logic live and explain failure paths.

AI Ethics, Tools & Collaboration

Tools: You must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for NoSQL.
AI Policy: AI usage must be documented. You are responsible for auditing AI-generated code for race conditions.
Team Work: 2–3 students per group. Task distribution must be clear across database layers.

Final Appendix: Tools & Methodology

1. **Software:** Students must use **Oracle SQL Developer / MS SQL Studio** for SQL and **MongoDB Compass** for Documents.
2. **Defense:** You must demonstrate the *Execution Plan* for your most complex query to prove optimization.
3. **AI Log:** Maintain a log of every AI prompt and the manual race-condition test performed to verify the code.
4. **Saga Diagram:** Provide a sequence diagram showing failure paths and compensating transactions for distributed logic.

– End of Assignment Proposal –